

Running Time

Running time measures how many basic operations an algorithm performs as a function of input size. Understanding running time is essential for predicting performance and choosing between algorithms.

Measuring Running Time

Operations counted: simple operations that take constant time (comparisons, arithmetic, array access, pointer dereference). Complex operations (sorting, searching) are counted by summing their basic steps.

Running time $T(n)$ is the number of basic operations for an input of size n .

Best, Average, Worst Case

Best case: fewest operations over all inputs of size n . - Example: linear search finds target at first position; $T(n) = O(1)$

Worst case: most operations over all inputs of size n . - Example: linear search looks for absent element; $T(n) = O(n)$

Average case: expected operations over random inputs of size n (requires probability model). - Example: hash table lookup with uniform hashing; $T(n) = O(1)$ expected

Worst case analysis is most common because it gives guarantees: no input can make the algorithm slower.

Asymptotic Analysis

Rather than exact counts (complex, input-dependent), we use asymptotic bounds to capture growth rates.

See [[Big-O, Big-Omega, Big-Theta]] for precise definitions.

Common bounds: - $O(1)$: constant (array access, hash lookup) - $O(\log n)$: logarithmic (binary search, balanced tree operations) - $O(n)$: linear (scanning array, DFS/BFS) - $O(n \log n)$: linearithmic (merge sort, efficient sorting) - $O(n^2)$: quadratic (nested loops; insertion sort, selection sort) - $O(2^n)$: exponential (subsets, exact algorithms for hard problems)

Trade-offs

Time vs space: some algorithms use extra memory to run faster (e.g., hash tables for $O(1)$ lookup instead of $O(n)$ scan).

Preprocessing: spend time upfront to enable faster queries (e.g., sort once, then binary search many times).

Amortized: individual operations may be expensive, but the average cost is low (e.g., dynamic array doubling; $O(1)$ amortized per push).

Practical Considerations

- Constants matter: $1000n$ may be slower than n^2 for small n
- Hidden factors: $O(n \log n)$ sorting may be faster than $O(n)$ counting sort depending on constants and cache behavior
- Real-world inputs: worst case may be rare; average or practical case is more relevant

But asymptotic analysis is the primary tool for algorithm design and comparison.

Related

- [[Big-O, Big-Omega, Big-Theta]]
- [[Space Complexity]]
- [[Amortized Analysis]]
- [[Recurrence Relations]]